OMN

Team 04 Final Presentation

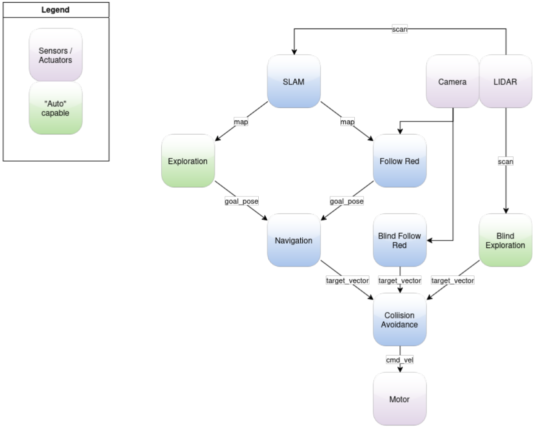
Robot Programing with ROS
Summer Semester 2026

Jun 03, 2026

Outline

Overview

Overview


Legend
Sensors / Actuators
"Auto" capable
scan
SLAM
Camera
LIDAR
map
map
Exploration
Follow Red
scan
goal_pose
goal_pose
Navigation
Blind Follow Red
Blind Exploration
target_vector
target_vector
target_vector
Collision Avoidance
cmd_vel
Motor

Exploration

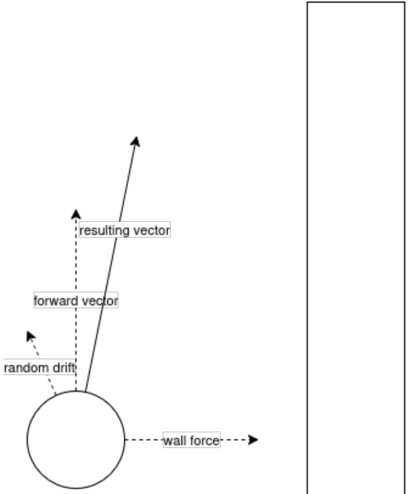
Blind Exploration

- No map given

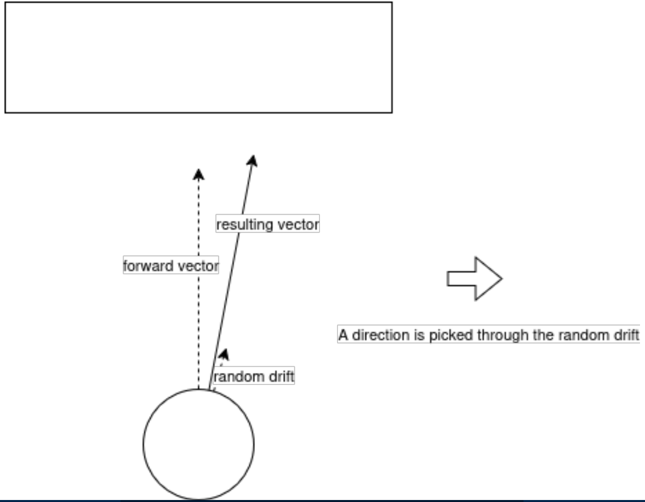
To complete the main goal *"Just keep moving!"* we do this:

1. Apply random drift
2. Drive forward
3. Wall detected?
   3.1 Yes: apply force to the side of the wall
4. Haven't moved for a while?
   4.1 Yes: apply randomized rotation

Blind Exploration



Blind Exploration - Why the drift?



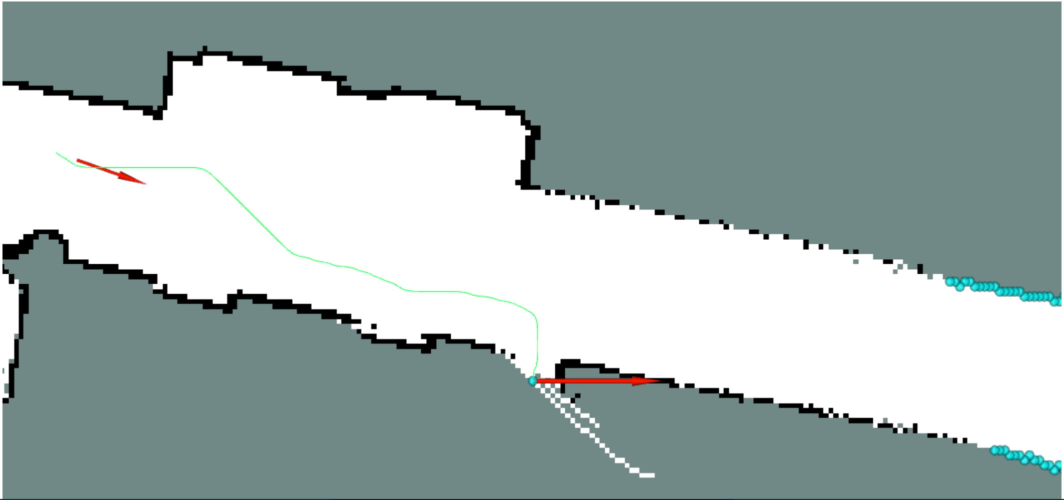
Exploration

- Sufficient exploration has taken place
- Map given

We now know where we are and where we've been, the goal changes to *"Explore the unknown!"*:

1. Flood fill the map from our current position
2. Create a frontier map
3. Select the closes frontier position (we haven't visited yet)
4. Send desired location to /goal_pose
5. Mark position as visited

Exploration

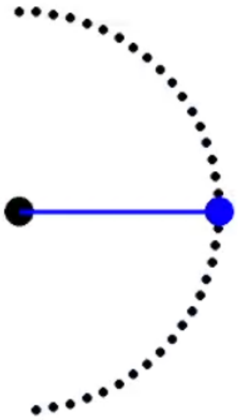


Collision Avoidance

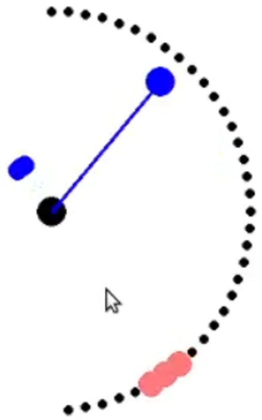
Collision Avoidance

- Has to be active if the robot wants to move
- Receives an /target_vector with a linear and angular velocity
- Ensures...
  - that the velocities are within safe bounds
  - that the movement direction does not point into obstacles
- Converts /target_vector to /cmd_vel and sends it off to motor

Collision Avoidance



Collision Avoidance



Navigation

Navigation Steps:

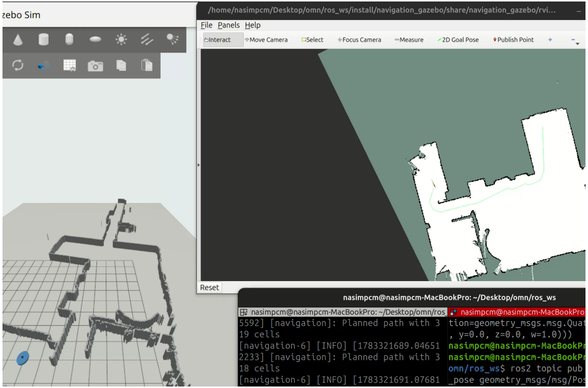
Planning

1. `timer_callback()` waits until a map, pose, and goal are available.
2. Robot and goal positions are converted from metres to map-grid cells.
3. A* plans a path using horizontal, vertical, and diagonal moves.
4. Obstacles are inflated to maintain clearance, and the grid path is smoothed to reduce zig-zag motion.

Path Following

5. Every $0.2\,\mathrm{s}$, a waypoint approximately $0.4\,\mathrm{m}$ ahead is selected.
6. A `TargetVector` pointing toward the waypoint is published.
7. When the goal is within $0.4\,\mathrm{m}$, the robot is stopped and `goal_succeeded` is reported.

A* Path Planning



Evaluation Function

$$f(n) = g(n) + h(n)$$

$g(n)$ is the path cost from the start and $h(n)$ estimates the remaining cost to the goal.

- Eight-connected grid search
- Straight-move cost: 1
- Diagonal-move cost: $\sqrt{2}$
- Corner-cutting is prevented
- Inflated cells add

A* Implementation

1. **Initialize:** add the start cell to the open-set priority queue, set $g(\text{start}) = 0$, and initialize `came_from`, `g_score`, and the closed set.
2. **Select:** remove the cell with the smallest estimated total cost $f(n)$.
3. **Check the goal:** if selected, reconstruct the path from the stored parent cells.
4. **Mark explored:** skip cells already processed; otherwise add the cell to the closed set.
5. **Generate neighbors:** consider all eight adjacent cells using straight and diagonal move costs.
6. **Apply obstacle costs:** reject blocked or invalid cells and penalize inflated cells near obstacles.
7. **Update better paths:** if a lower cost is found, update the neighbor's parent and scores, then add it to the open set.
8. **Finish:** return the reconstructed path, or report `no_path` if the open set becomes empty.

Next Waypoint Selection

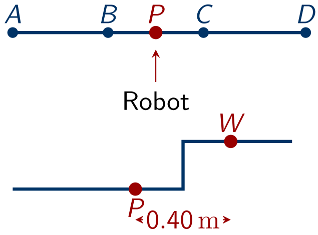
1. Locate the Robot on the Path

Project the robot onto every remaining path segment and select the closest point P.

2. Look Ahead Along the Path

Starting at P, measure $0.40\,\mathrm{m}$ along the path to obtain waypoint W. If the distance crosses a corner, continue on the next segment.

- `path_progress` prevents the selected position from jumping backward when the path passes close to itself.
- If less than $0.40\,\mathrm{m}$ remains, return the final



Look-ahead continues across corners

Follow Red

Goal in 3 Steps

1. Detect a red cube via OpenCV
2. Estimate its distance/angle or global map position
3. Publish targets in a ROS 2 system

- **Node Type:** ROS 2 Lifecycle Node
- **Input:** OpenCV camera stream
- **Processing:** HSV masking, exponential smoothing, median filtering
- **Output:** Polar coordinates (`TargetVector`) or global map poses (`PoseStamped`)

Given Hardware and Calibration

Type	Webcam
Frame	640x480
FOV	60
Calibrated Distance	1 Meter
Calibrated Pixels	9750

Dual Target Publishing Modes

Controlled by the runtime parameter `send_polar`:

Polar Mode (`send_polar = True`)

- Publishes `TargetVector` to `/target_vector`.
- Direct values for `linear` (distance) and `angular` (bearing angle).

Global Pose Mode (`send_polar = False`)

- Uses `tf2_ros` for transform lookups (`map` $\rightarrow$ `base_footprint`).
- Transforms camera relative coordinates into global map coordinates.
- Publishes `PoseStamped` to `/goal_pose`.

Image Processing & Filter Pipeline

1. **Filter Size:**
   - Accept only a sum over 500px as possible cube.
2. **Area Change Rate Filter (Jump Filter):**
   - Ignores frames with sudden, extreme area size spikes ($\frac{A_{\text{new}}}{A_{\text{old}}} \notin [\min, \max]$).
   - Prevents position jumps caused by brief reflections or partial occlusions.
3. **HSV Masking:**
   - Dual range masking for red hue due to HSV wrap-around ($0^\circ - 5^\circ$ and $174^\circ - 179^\circ$).
4. **Low-Pass Filter (Exponential Smoothing):**
   - Noise reduction for pixel position ($\alpha_{\text{pos}} = 0.25$) and area ($\alpha_{\text{area}} = 0.20$).
5. **Median Ring Buffer:**
   - `deque(maxlen=5)` eliminates sporadic false detections over 5 frames.
6. **Distance Estimation:**
   - Inverse square-root calculation based on calibrated pixel area:

$$d = d_0 \cdot \sqrt{\frac{A_0}{A}}$$

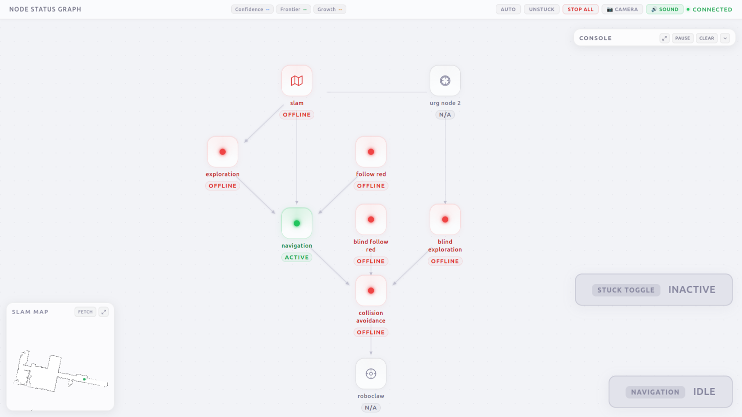
Fault Tolerance

Automatic Camera Reconnect

- Monitors consecutive read errors
- When reaching `CAMERA_MAX_ERRORS` (40 frames $\approx 2\,\mathrm{s}$), the video device is released and re-initialized.

Manager

Dashboard


NODE STATUS GRAPH
Confidence –
Frontier –
Growth –
AUTO
UNSTUCK
STOP ALL
CAMERA
SOUND
CONNECTED
CONSOLE
PAUSE
CLEAR
slam
OFFLINE
urg node 2
N/A
exploration
OFFLINE
follow red
OFFLINE
navigation
ACTIVE
blind follow red
OFFLINE
blind exploration
OFFLINE
collision avoidance
OFFLINE
roboclaw
N/A
SLAM MAP
FETCH
STUCK TOGGLE
INACTIVE
NAVIGATION
IDLE

Navigation (stuck, smoothing, theta* v a*)

Path Smoothing via LOS

LOS-Algorithm

- Path from A* gets checked for shortcuts
- Line of Sight is used
- starts from first point, checks for 50 cells ahead
- looks at furthest away, moves closer until visible
- repeat for the next point

Reasons for/against using

- For: Very smooth path
- For: Decreased number of waypoints
- For: Changable Look-Ahead-Distance
- Against: High computational Cost
- Against: Too slow for constant path replanning
- Result: Not used

Path Smoothing via Gradient

Algorithm

- Path from A* gets corrected
- Multiple passes
- each coordinate gets tied to its origin
- and gets pulled away by: y_next + y_prev - 2.0 * y_curr
- stops automatically after specified passes or little change occuring

Reasons for/against using

- For: Low Computational Cost
- For: Tweakable
- Against: Path not as smooth
- Against: Might drag path through walls
- Result: Used

Stuck Logic

Algorithm

- only when in mode "tracking" to avoid timeout while calculating path
- checks weather the robot has moved enough in a specified timeframe
- when stuck, clears the current path and publishes velocities $= 0$
- when next_goal in queue, switch over

Toggle Logic

set toggle to false, when:

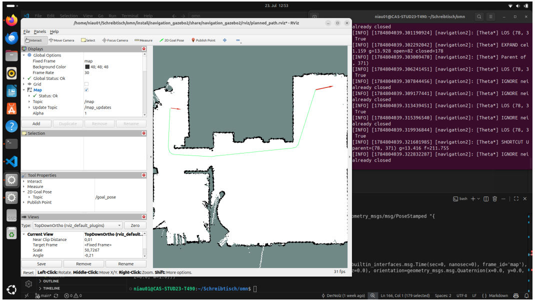
- goal is reached or new current goal detected
- stuck and there is a next goal
- path is cleared, there is no path or a new path was created
- not ready to plan (no pose, no map or no goal)
- replanning

set toggle to true, when:

- following a path

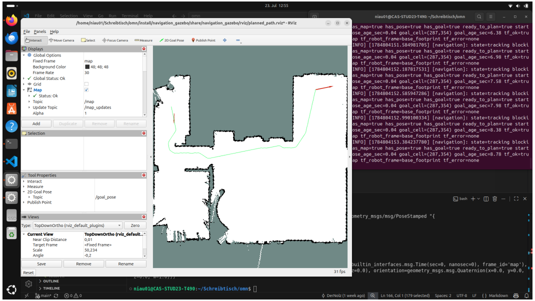
Theta* vs A*

Theta*



Time of Computation: approx. 240 Seconds Noticably smoother path

A*



Time of Computation: approx. 2 seconds Rougher path even after path smoothing

==> A* wins due to faster computation

Goal Queue: Problem and Solution

Previous Behaviour

- Navigation handled only one goal
- New goal replaced the active goal
- Older goals were discarded

New Behaviour

- Added FIFO goal queue
- Current goal stays active
- New goals wait in queue
- Goals follow received order

FIFO Order

Goal A $\rightarrow$ Goal B $\rightarrow$ Goal C

Goal Queue: Main Logic

When a New Goal Arrives

1. Transform the goal into the map frame
2. Check whether it is a duplicate
3. Activate it if no goal is active
4. Otherwise, append it to the queue
5. Reject it if the queue is full

Queue Settings

- Minimum spacing: 0.25 m
- Maximum size: 100 goals
- New goals added with:

  `append()`

- Next goal taken with:

  `popleft()`

Code Added In

- Import section and `__init__()`
- `goal_pose_callback()`
- New `activate_next_goal()` method
- Goal-reached part of `timer_callback()`

Goal Queue: Simulation Result

Waypoint Queued

```
[navigation-7] [INFO] [1783806059.833257233] [navigation]: Planned path with 132 cells; reason=robot off path
[navigation-7] [INFO] [1783806059.842346483] [navigation]: Got new position
[navigation-7] [INFO] [1783806060.367907720] [navigation]: Got new position
[navigation-7] [INFO] [1783806061.099720824] [navigation]: Got new position
[navigation-7] [INFO] [1783806061.914410483] [navigation]: Got new position
[navigation-7] [INFO] [1783806062.728160352] [navigation]: Planned path with 131 cells; reason=robot off path
[navigation-7] [INFO] [1783806062.739694201] [navigation]: Got new position
[navigation-7] [INFO] [1783806063.211910323] [navigation]: Got new position
[rviz2-9] [INFO] [1783806063.389456667] [rviz2]: Setting goal pose: Frame:map, Position(13.7475, -7.74069, 0), Orientation(0, 0, 0.996266, 0.0863369) = Angle: 2.9687
[navigation-7] [INFO] [1783806063.390926198] [navigation]: Queued goal x=13.747, y=-7.741; queue size=3
[navigation-7] [INFO] [1783806063.392111188] [navigation]: NAVIGATION STATUS | state=WAYPOINT_QUEUED | goal=(5.17, -6.54) | queued=3 | Waypoint added to the navigation queue
[navigation-7] [INFO] [1783806063.877134359] [navigation]: NAVIGATION STATUS | state=MOVING | goal=(5.17, -6.54) | queued=3 | Following path to active waypoint
[navigation-7] [INFO] [1783806063.877882953] [navigation]: Got new position
[navigation-7] [INFO] [1783806064.576573798] [navigation]: Got new position
[navigation-7] [INFO] [1783806065.419140141] [navigation]: Got new position
[navigation-7] [INFO] [1783806066.045373230] [navigation]: Got new position
[navigation-7] [INFO] [1783806066.792473817] [navigation]: Planned path with 129 cells; reason=robot off path
[navigation-7] [INFO] [1783806066.798104245] [navigation]: Got new position
[navigation-7] [INFO] [1783806067.220953412] [navigation]: Got new position
[navigation-7] [INFO] [1783806067.832294191] [navigation]: Got new position
[navigation-7] [INFO] [1783806068.391624387] [navigation]: Got new position
[navigation-7] [INFO] [1783806069.267973276] [navigation]: Planned path with 128 cells; reason=robot off path
[navigation-7] [INFO] [1783806069.272062410] [navigation]: Got new position
[navigation-7] [INFO] [1783806069.720720578] [navigation]: Got new position
[navigation-7] [INFO] [1783806070.491743339] [navigation]: Got new position
[navigation-7] [INFO] [1783806071.212280124] [navigation]: Got new position
[navigation-7] [INFO] [1783806072.046441321] [navigation]: Planned path with 127 cells; reason=robot off path
```

- Queue size reached 3
- Active goal was not replaced

Queue Completed

```
[navigation-7] [INFO] [1783806770.506409306] [navigation]: Planned path with 4 cells; reason=robot off path
[navigation-7] [INFO] [1783806770.511580410] [navigation]: Got new position
[navigation-7] [INFO] [1783806771.088161238] [navigation]: Got new position
[navigation-7] [INFO] [1783806771.643636478] [navigation]: Planned path with 3 cells; reason=robot off path
[navigation-7] [INFO] [1783806771.644610162] [navigation]: Got new position
[navigation-7] [INFO] [1783806772.201378181] [navigation]: Got new position
[navigation-7] [INFO] [1783806772.737215545] [navigation]: Waypoint reached x=13.747, y=-7.741
[navigation-7] [INFO] [1783806772.737799274] [navigation]: NAVIGATION STATUS | state=WAYPOINT_REACHED | goal=(13.75, -7.74) | queued=0 | Reached waypoint (13.747, -7.741)
[navigation-7] [INFO] [1783806772.738478984] [navigation]: NAVIGATION STATUS | state=IDLE | goal=none | queued=0 | All queued waypoints completed
[navigation-7] [INFO] [1783806772.738862275] [navigation]: All queued goals succeeded
[navigation-7] [INFO] [1783806772.739373254] [navigation]: NAVIGATION STATUS | state=GOAL_REACHED | goal=none | queued=0 | Complete waypoint queue executed
[navigation-7] [INFO] [1783806772.739992372] [navigation]: Got new position
[navigation-7] [INFO] [1783806773.286649384] [navigation]: Got new position
```

- Queue returned to 0
- All queued goals succeeded

Result

Goals are executed sequentially and are no longer discarded.

Localization (in Sim)

Localization (in Sim)

- essientially dynamic obstacle avoidance